

Trabajo Práctico Diseño

Money Laundering Analysis

25/06/2026

1º cuatrimestre 2026

Sistemas Distribuidos I (TA050)

Cátedra: Roca



Thiago Fernando Baez
110703
tbaez@fi.uba.ar

Valentina Llanos Pontaut
104413
vllanos@fi.uba.ar

Índice

Trabajo Práctico Diseño.....	0
Índice.....	1
Documento de Diseño de Arquitectura.....	3
DAG.....	3
Vista de Escenarios.....	5
Diagrama de Casos de Uso.....	5
Vista Física.....	6
Diagrama de Robustez.....	6
Diagrama General.....	6
Query 1 y Query 2.....	6
Query 3 — Transacciones menores al 1% del promedio del período anterior.....	8
Query 4 — Patrón Scatter-Gather.....	8
Query 5 — Cantidad de transacciones Wire/ACH menores a 1 USD.....	9
Diagrama de Despliegue.....	11
Componentes.....	11
Vista General.....	11
Worker Q3.....	12
Workers Q4.....	13
Vista de Desarrollo.....	13
Diagrama de Paquetes.....	13
Estructura.....	13
Agrupamiento de workers.....	14
Vista de procesos.....	14
Diagrama de Actividades.....	14
Flujo principal.....	15
Query 1 — Transacciones USD menores a 50.....	15
Query 2 — Máxima transacción USD por banco.....	16
Query 3 — Transacciones menores al 1% del promedio del período anterior.....	17
Query 4 — Patrón Scatter-Gather.....	18
Query 5 — Cantidad de transacciones Wire/ACH menores a 1 USD.....	19
Diagramas de Secuencia.....	20
Consulta 1 - Transacciones USD menores a 50.....	20
Consulta 2 - Máxima transacción USD por banco.....	21
Consulta 3 - Transacciones menores al 1% del promedio.....	22
Consulta 4 - Patrón Scatter-Gather.....	22
Consulta 5 - Cantidad de transacciones Wire/ACH menores a 1 USD.....	23
Tolerancia a Fallos.....	24
Watchdog y heartbeats.....	24
Checkpoints y deduplicación.....	24
Disk spill atómico y recuperación.....	24
Garantía resultante.....	25
Backlog.....	26

Documento de Diseño de Arquitectura

El presente documento describe la arquitectura de un sistema distribuido orientado al análisis de transacciones financieras para la detección de patrones asociados al lavado de dinero, tales como fan-in, fan-out y scatter-gather. Está organizado siguiendo el modelo de vistas 4+1, que estructura la arquitectura en cinco perspectivas complementarias:

- **Vista de escenarios (+1):** describe los casos de uso principales que guían y validan las decisiones arquitectónicas. Representada por el *diagrama de casos de uso*.
- **Vista física:** describe la distribución del sistema en contenedores y cómo se comunican entre sí. También muestra la interacción y conexión entre los distintos actores del sistema. Representada por el *diagrama de despliegue y el de robustez*.
- **Vista de desarrollo:** describe la estructura del código fuente y cómo está organizado para su implementación. Representada por el *diagrama de paquetes*.
- **Vista lógica:** describe las entidades del dominio, sus atributos y relaciones. Representada por el *diagrama de clases*. Dado que el sistema aún no ha sido implementado, esta vista se omite en la presente entrega y será incorporada en una iteración futura.
- **Vista de proceso:** describe el flujo de control, el paralelismo y la interacción entre componentes en tiempo de ejecución. Representada por los *diagramas de actividades y de secuencia*.

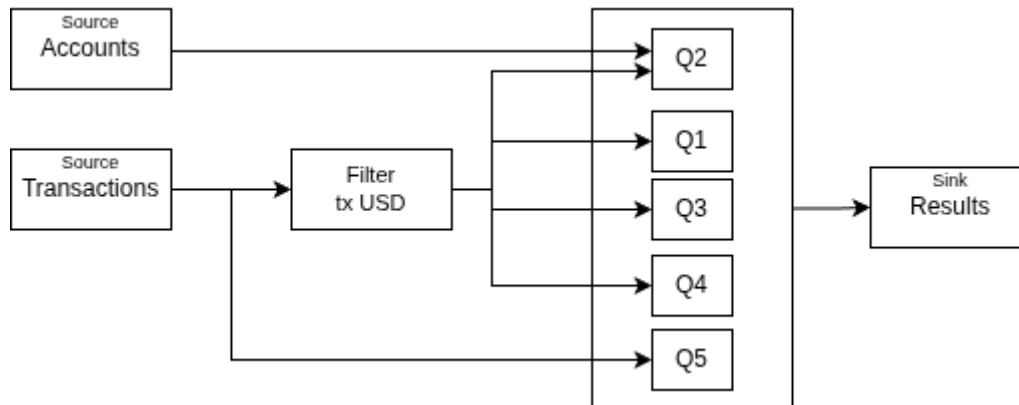
Adicionalmente se incluye un DAG que modela las dependencias entre etapas del pipeline de procesamiento.

El sistema procesa grandes volúmenes de transacciones de forma distribuida, escalando horizontalmente mediante la replicación de workers y garantizando tolerancia a fallos mediante watchdogs que detectan y reinician containers caídos, con soporte de graceful shutdown ante señales SIGTERM. Los datos de entrada provienen de un dataset de simulación de transacciones financieras de la plataforma Kaggle, complementado con valores de cotización diaria obtenidos de la API de Frankfurter para la conversión de divisas a USD.

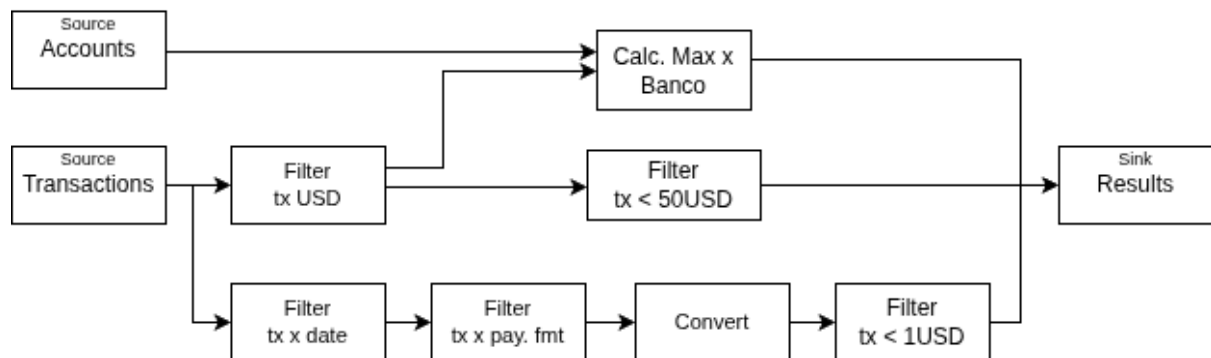
DAG

El DAG (Directed Acyclic Graph) modela las dependencias entre etapas del pipeline de procesamiento. No muestra flujo de control ni mensajes — muestra qué etapas deben completarse antes de que otra pueda comenzar.

A continuación, se muestra el flujo general: las fuentes Transactions y Accounts alimentan el pipeline. Las tx pasan por un Filter tx USD antes de llegar a Q1, Q2, Q3 y Q4. Q5 consume directamente de Transactions sin pasar por ese filtro, ya que necesita tx en cualquier moneda. Todas las queries producen resultados que convergen en Sink Results.

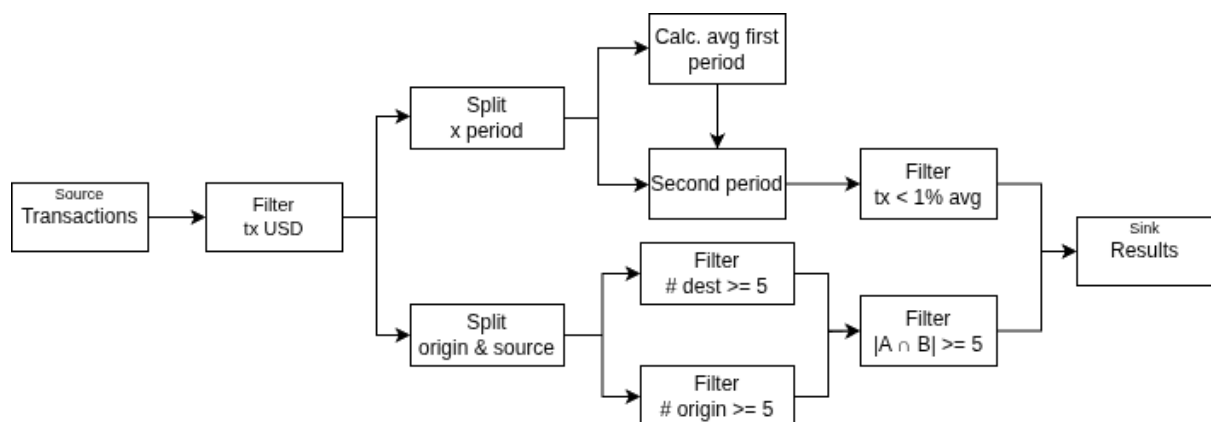


El siguiente diagrama detalla Q1, Q2 y Q5. Q1 aplica un Filter tx < 50 USD y publica directo al sink. Q2 calcula el máximo por banco y lo joinea con Accounts. Q5 aplica tres etapas en cadena: Filter tx x date -> Filter tx x pay. fmt -> Convert -> Filter tx < 1 USD.



Por último, este diagrama detalla Q3 y Q4:

- Q3 hace un Split x period — el período 1 alimenta Calc. avg first period y el período 2 espera ese promedio para pasar por Filter tx < 1% avg.
- Q4 hace un Split origin & source que alimenta en paralelo Filter # dest >= 5 y Filter # origin >= 5, cuyos resultados convergen en Filter $|A \cap B| \geq 5$.



Vista de Escenarios

Diagrama de Casos de Uso

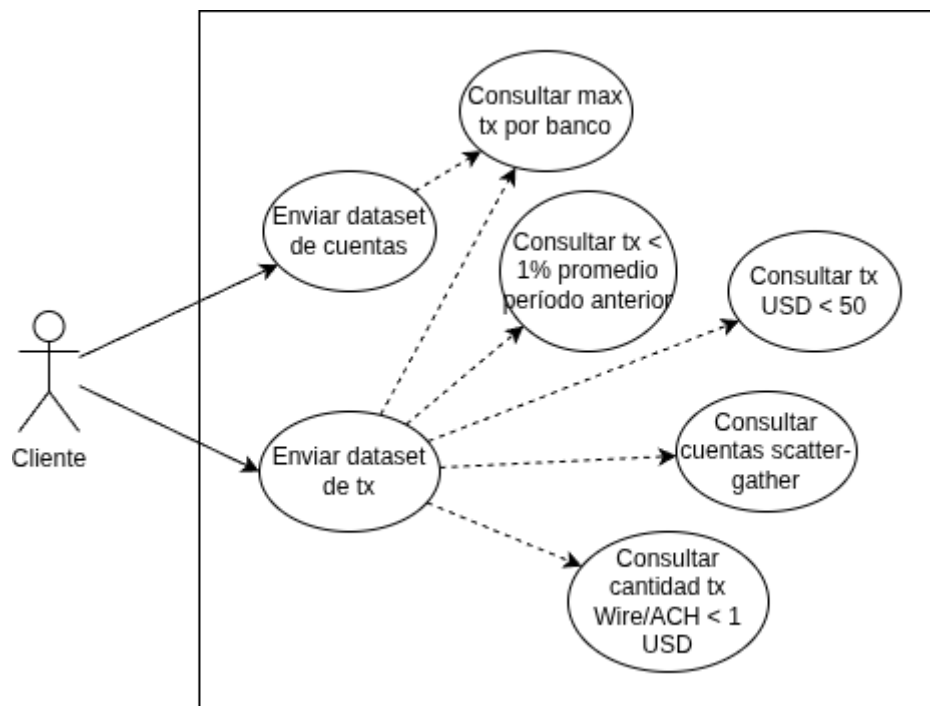
El diagrama de casos de uso describe las interacciones entre el sistema y sus actores externos. En este sistema el único actor es el Cliente, que interactúa directamente con el sistema a través de dos casos de uso:

- Enviar dataset de cuentas — el cliente envía el CSV con la información de cuentas bancarias (A_i).
- Enviar dataset de transacciones — el cliente envía el CSV con las transacciones financieras (T_i).

Como consecuencia del envío de los datasets, el sistema ejecuta automáticamente las 5 consultas:

- Consultar tx USD < 50
- Consultar max tx por banco — extiende también el envío de cuentas, ya que requiere el bank_name para resolver el resultado.
- Consultar tx < 1% promedio período anterior
- Consultar cuentas scatter-gather
- Consultar cantidad tx Wire/ACH < 1 USD

Los dos casos de uso directos ocurren en una única sesión — el cliente no puede solicitar las consultas de forma independiente ni parcial.



Vista Física

Diagrama de Robustez

El diagrama de robustez modela el flujo de procesamiento del sistema, mostrando la interacción entre actores, objetos de borde, de control y de entidad. Su propósito es evidenciar la separación de responsabilidades y el pipeline de procesamiento distribuido.

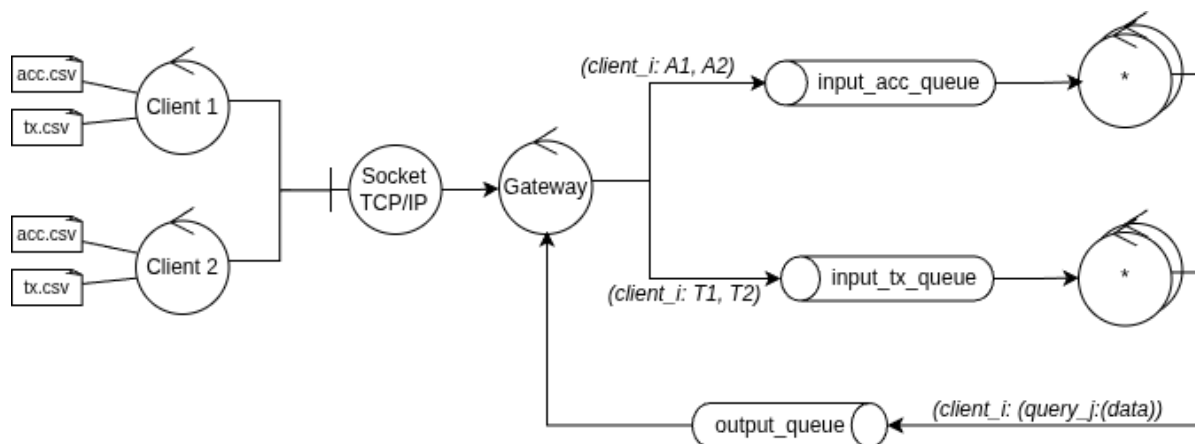
El diagrama se presenta en partes: primero una vista general del sistema que muestra el flujo de entrada y salida; luego cada query se descompone en su propio diagrama con el detalle de sus workers internos.

Diagrama General

El sistema recibe como entrada dos archivos por cliente: un CSV de transacciones y un CSV de cuentas bancarias. Ambos son enviados al gateway vía TCP/IP, que los separa en dos colas: **input_acc_queue** para las cuentas e **input_tx_queue** para las transacciones.

Los resultados de cada query se publican directamente en **output_queue** a medida que se producen. El gateway escucha de esta cola y devuelve cada resultado al cliente correspondiente vía TCP/IP.

El detalle de cómo cada query procesa las transacciones se desarrolla en los diagramas siguientes.



Query 1 y Query 2

Q1 — Transacciones USD menores a 50

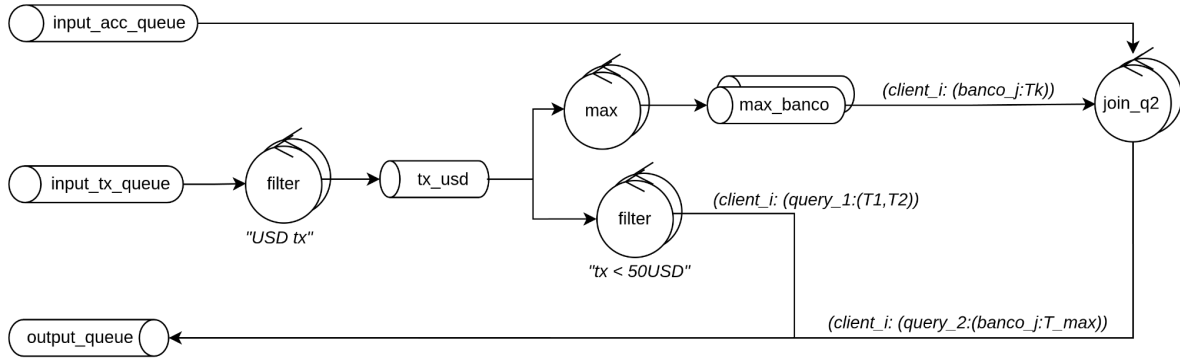
Un filter consume de `input_tx_queue` y retiene sólo las transacciones en USD, publicándolas en `tx_usd`. Un segundo filter consume de `tx_usd_conv` y retiene solo las tx con `amount_usd < 1`, publicando las transacciones que cumplen directamente en `output_queue`:

`(client_i: (query_1: (T1.orig, T1.dest, T1.monto)))`

Las que no cumplen se descartan.

Query 2 — Máxima transacción USD por banco

El primer filter es compartido con Q1 — publica en `tx_usd`. Los workers `max` compiten por esta cola, cada uno acumula `{banco_j: max_tx}` para el subconjunto de transacciones que le tocan.



Join con accounts en join_q2: consume datos de dos fuentes: **max_banco**, que contiene los máximos parciales calculados por los workers, e **input_acc_queue**, que contiene la información de las cuentas bancarias.

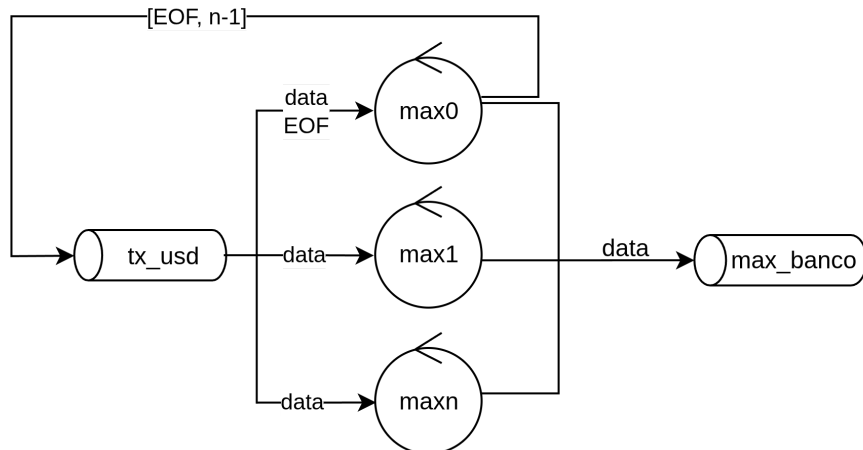
Los máximos parciales se distribuyen utilizando un hash del bank_id, garantizando que todos los máximos correspondientes a un mismo banco sean enviados a la misma instancia de join_q2. De esta manera, cada join puede desempatar localmente entre los máximos parciales recibidos para un mismo banco.

Una vez determinado el máximo final, join_q2 realiza el join entre bank_id y bank_name. Dado que el volumen de cuentas bancarias es significativamente menor al de transacciones (~100.000 vs ~10.000.000 por día), se asume que la información de accounts llega primero. Sin embargo, join_q2 verifica explícitamente la recepción del EOF de accounts antes de comenzar el proceso de join.

El resultado final se publica en output_queue con el siguiente formato:

(client_i: (query_2: (bank_name: (T1.origen, T1.monto))))

Propagación de EOF entre workers max: el primer worker que recibe el EOF lo reencola en input_queue junto con un contador inicializado con la cantidad de workers. Cada worker que consume el EOF y aún no lo había procesado decrementa dicho contador y vuelve a publicar el mensaje en input_queue, permitiendo que el EOF circule entre todos los workers. Cuando el último worker recibe el EOF y el contador llega a cero, publica el EOF en la output_queue, indicando que todos los workers finalizaron el procesamiento y ya emitieron sus resultados parciales (client_i: {banco_j: (T1, T2)}) en max_banco.



Query 3 — Transacciones menores al 1% del promedio del período anterior

El worker **split** consume de tx_usd y divide las tx en dos ramas según fecha:

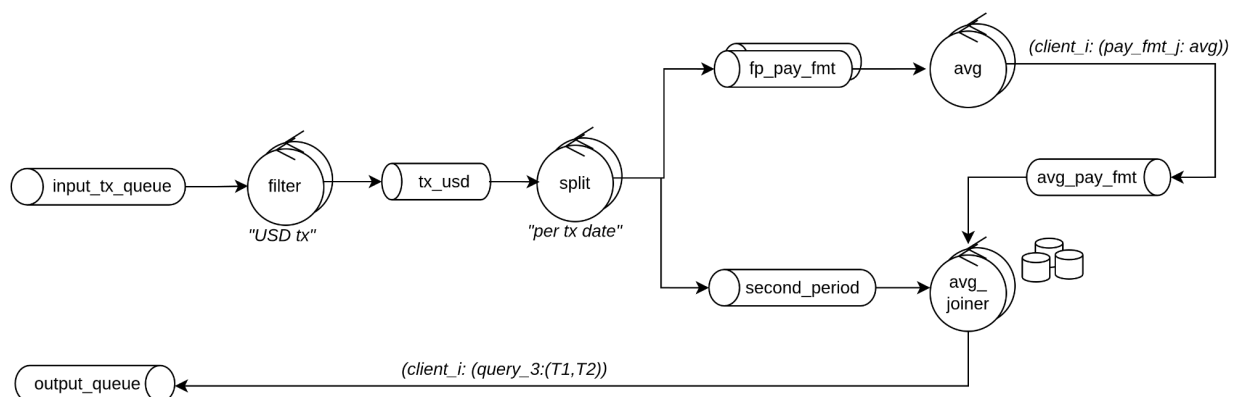
- *Período 1 [2022-09-01, 2022-09-05]*: las transacciones se publican directamente en distintas colas según el hash del atributo payment_fmt, garantizando que todas las transacciones de un mismo formato de pago sean procesadas por el mismo worker avg.

Los workers avg consumen de estas colas y acumulan suma y count para los formatos de pago que les corresponden. Al recibir el EOF, cada worker calcula el promedio y publica (client_i: {pay_fmt_j: avg_k}) en avg_pay_fmt. Luego, mediante un exchange fanout, estos resultados se distribuyen a todos los workers avg_joiner, que reciben simultáneamente los promedios de todos los formatos de pago.

- *Período 2 [2022-09-06, 2022-09-15]*: las tx se publican en **second_period** y son consumidas aleatoriamente por los workers **avg_joiner**. Cada worker persiste sus tx en un volumen en disco particionado por client_id y payment_fmt — este es el mecanismo de spill to disk detallado en la sección de despliegue.

A medida que llegan los promedios via fanout, cada **avg_joiner** procesa las tx almacenadas para ese formato de pago y evalúa si amount_usd < 1% del promedio — las que cumplen se publican en **output_queue**, las que no se descartan:

(client_i: (query_3: (T1.origen, T1.monto)))

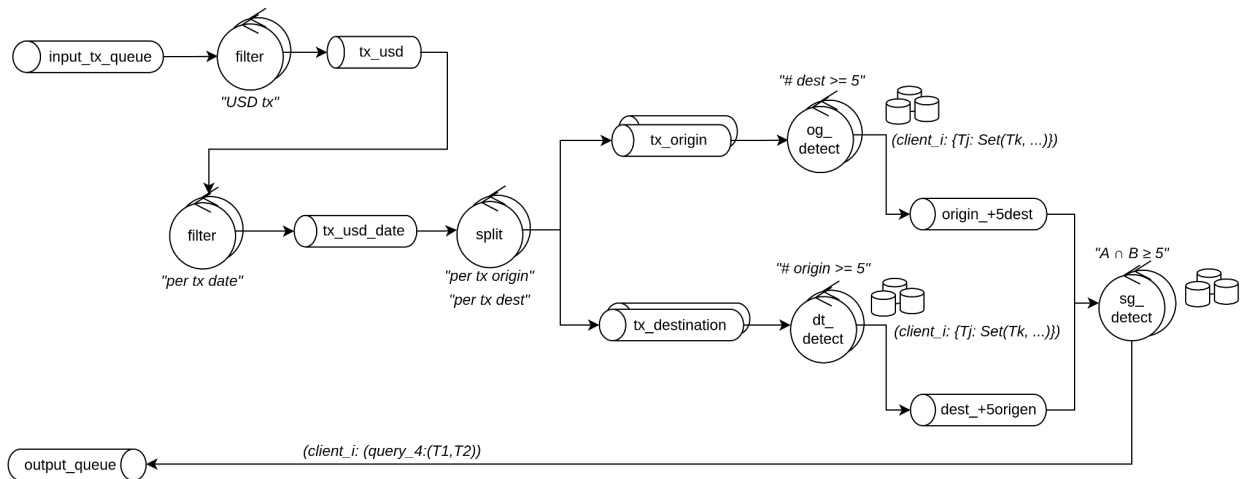


Query 4 — Patrón Scatter-Gather

Un filter consume de tx_usd y retiene sólo las tx del período [2022-09-01, 2022-09-05], publicándolas en una cola **tx_usd_date**. El worker **split** procesa cada tx y la publica en dos ramas simultáneamente:

- Hashea el account_origin y publica en la queue de **tx_origin** correspondiente.
- Hashea el account_dest y publica en la queue de **tx_destination** correspondiente.

Esto garantiza que todas las tx de un mismo origen lleguen siempre al mismo worker **og_detect**, y todas las tx de un mismo destino lleguen siempre al mismo worker **dt_detect** — sin necesidad de coordinación entre instancias.



Rama origen (og_detect): cada worker persiste en disco, por cliente y cuenta origen, el conjunto de cuentas destino a las que se enviaron transacciones. Al recibir el EOF, filtra únicamente las cuentas origen con 5 o más destinos distintos y publica los resultados en *origin_+5dest*.

Rama destino (dt_detect): de manera simétrica, cada worker persiste en disco, por cliente y cuenta destino, el conjunto de cuentas origen desde las que recibió transacciones. Al EOF, filtra únicamente las cuentas destino con 5 o más orígenes distintos y publica los resultados en *dest_+5origin*.

sg_detect: consume información de ambas ramas. Los conjuntos provenientes de *dest_+5origin* se propagan a todos los workers mediante un exchange fanout, mientras que los conjuntos provenientes de *origin_+5dest* se distribuyen utilizando un hash sobre el nombre de la cuenta origen, garantizando que cada cuenta sea procesada siempre por el mismo worker *sg_detect*.

Cada worker calcula la intersección entre los destinos de una cuenta origen candidata y los orígenes asociados a cada cuenta destino candidata. Si la intersección cumple $|A \cap B| \geq 5$, se confirma el patrón scatter-gather y se publica el resultado directamente en *output_queue*: *(client_i: (query_4: [account_1, account_2, ...]))*

Query 5 — Cantidad de transacciones Wire/ACH menores a 1 USD

A diferencia de las otras queries, Q5 consume directamente de **input_tx_queue** ya que necesita tx en cualquier moneda.

Los filtros se aplican en cadena priorizando los que más volumen descartan primero:

1. Un filter filtra por fecha y formato de pago, reteniendo solo las tx del período [2022-09-01, 2022-09-05] con formato Wire o ACH y publicando en **tx_esp_date**.
2. El converter consume de *tx_esp_day*, convierte el monto a USD consultando la Currency Exchange API y publica en **tx_usd_conv**.
3. Un segundo filter consume de *tx_usd_conv* y retiene solo las tx con *amount_usd* < 1, publicando el conteo directamente en **output_queue**:

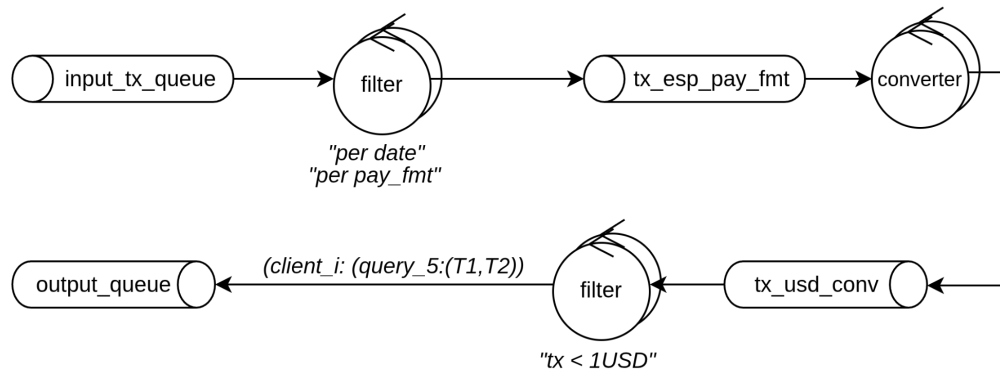


Diagrama de Despliegue

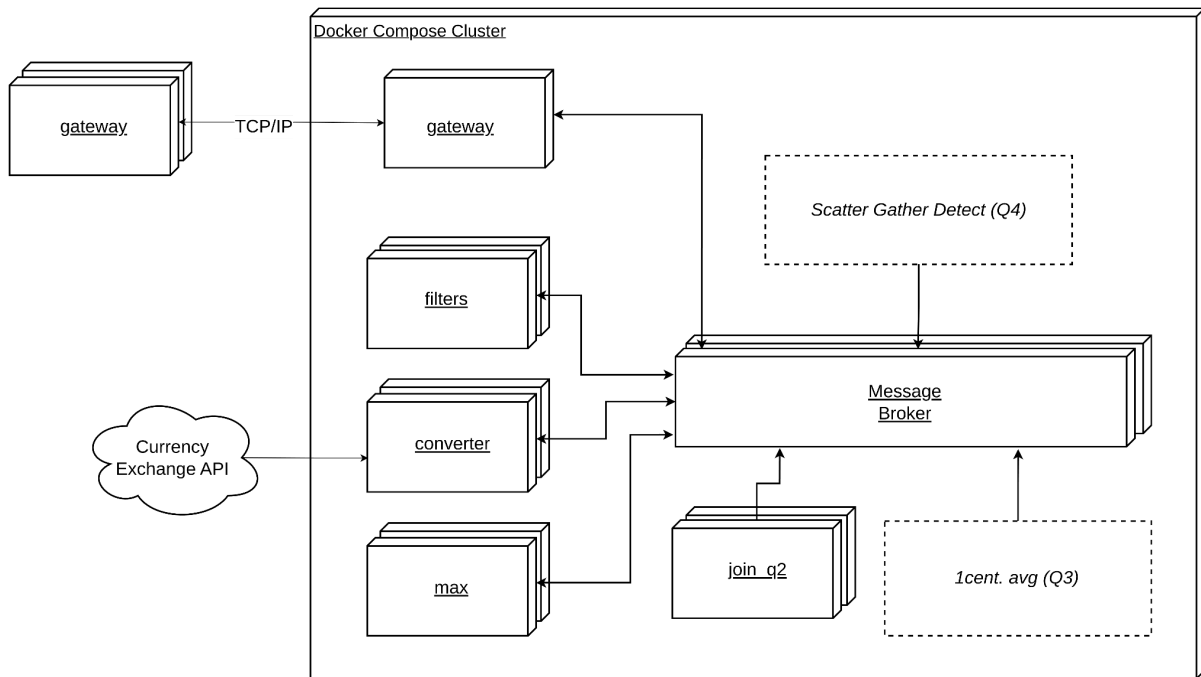
El diagrama de despliegue modela la distribución física del sistema en contenedores Docker, mostrando qué procesa cada nodo y cómo se comunican entre sí. Se presenta en dos partes: una vista general del sistema y el detalle de los workers de Q3 y Q4.

Componentes

Contenedor	Multiplicidad	Rol
gateway	1	Punto de entrada TCP/IP. Recibe datasets de tx y cuentas y publica en el Message Broker
Message Broker	1..N	Gestiona todas las colas entre workers
filters	1..N	Filtran tx por criterio específico según configuración — moneda, fecha, formato de pago o monto. Pueden existir múltiples workers con diferentes configuraciones
converter	1..N	Convierte monto de cada tx a USD consultando la Currency Exchange API
max	1..N	Calcula tx de mayor monto por banco
join_q2	1..N	Desempata máximos parciales entre workers max y hace join con accounts
split	1..N	Divide flujo de tx por origen y destino para Q4
split_date	1..N	Divide flujo de tx por periodos de fecha para Q3
avg	1..N	Calcula promedio de tx del período 1 por formato de pago
avg_joiner	1..N	Combina tx período 2 con promedio período 1
og_detect	1..N	Detecta cuentas origen con 5+ destinos distintos
dt_detect	1..N	Detecta cuentas destino con 5+ orígenes distintos
sg_detect	1..N	Detecta patrón scatter-gather por intersección de conjuntos
watchdog	1..N	Monitorea workers via heartbeats y los reinicia ante caídas usando la API de Docker

Los workers marcados con [1..N] pueden replicarse agregando instancias al Docker Compose. Al competir por la misma cola del Message Broker, éste distribuye los mensajes automáticamente entre instancias sin coordinación adicional. Los workers [1] son únicos por diseño: sg_detect necesita ver la totalidad de los conjuntos para calcular la intersección, y join_q2 debe consolidar todos los máximos parciales antes de poder responder.

Vista General

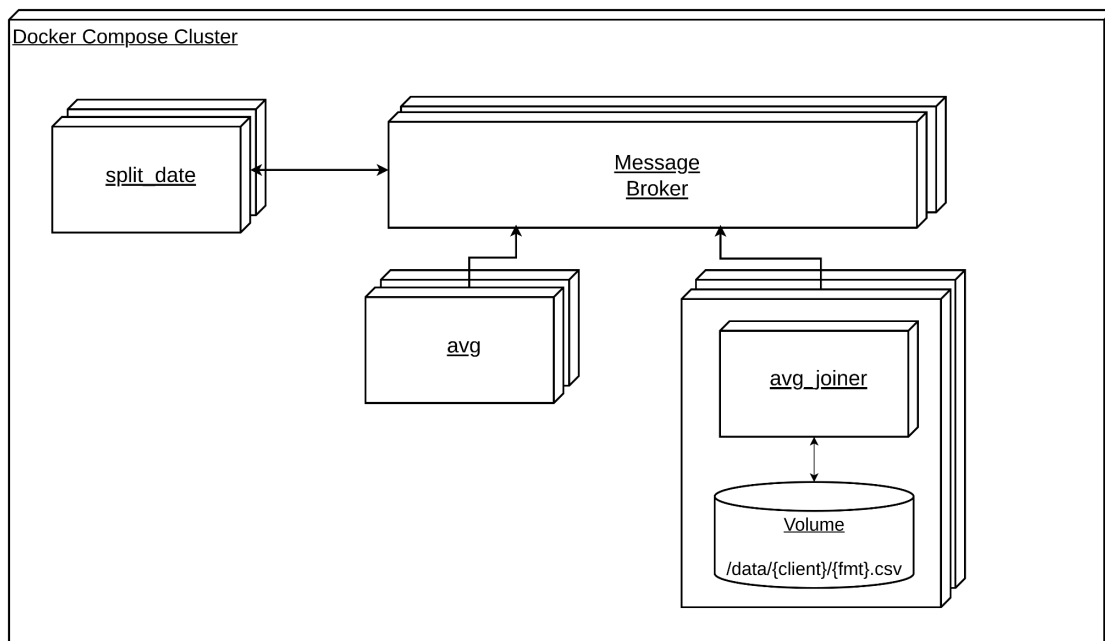


El sistema corre en un Docker Compose Cluster con un Message Broker como componente central que intermedia toda la comunicación entre workers — ningún componente se comunica directamente con otro.

Los clientes corren fuera del cluster y se conectan al gateway por TCP/IP. El converter es el único componente que además se comunica con un servicio externo: la Currency Exchange API para obtener tipos de cambio actualizados.

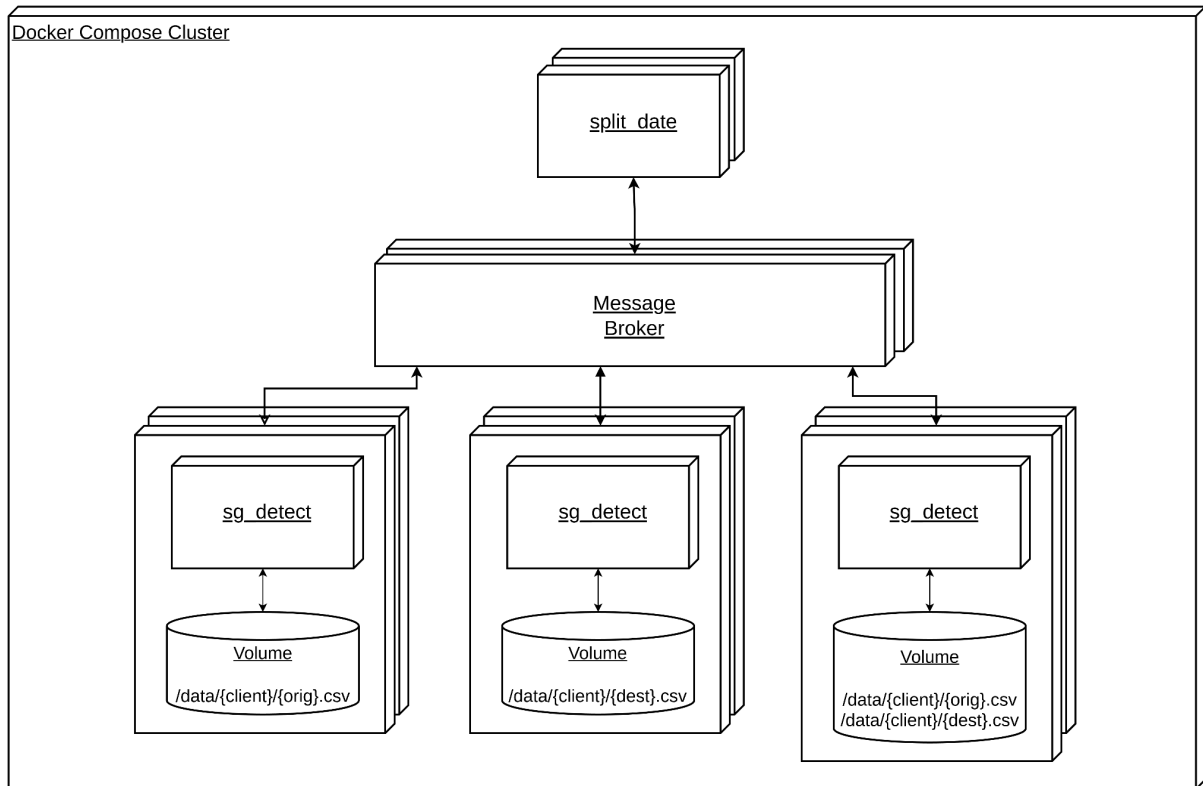
Worker Q3

El avg_joiner persiste datos en un volumen dedicado por instancia con la estructura `/data/{client}/{fmt}.csv`. Todos los demás workers se comunican exclusivamente a través del Message Broker.



Workers Q4

Cada worker de la query 4 persiste datos en un volumen dedicado por instancia con la estructura `/data/{client}/{fmt}.csv`.



Vista de Desarrollo

Diagrama de Paquetes

El diagrama de paquetes modela la organización lógica del código — cómo está estructurado en módulos, qué depende de qué y qué está agrupado junto. A diferencia de los diagramas anteriores que muestran flujo o infraestructura, este diagrama representa la arquitectura del software en sí.

Estructura

El proyecto se organiza en dos grandes áreas: la capa de infraestructura (client, gateway, common) y la capa de negocio (business). Todos los paquetes dependen de common, que centraliza la lógica compartida:

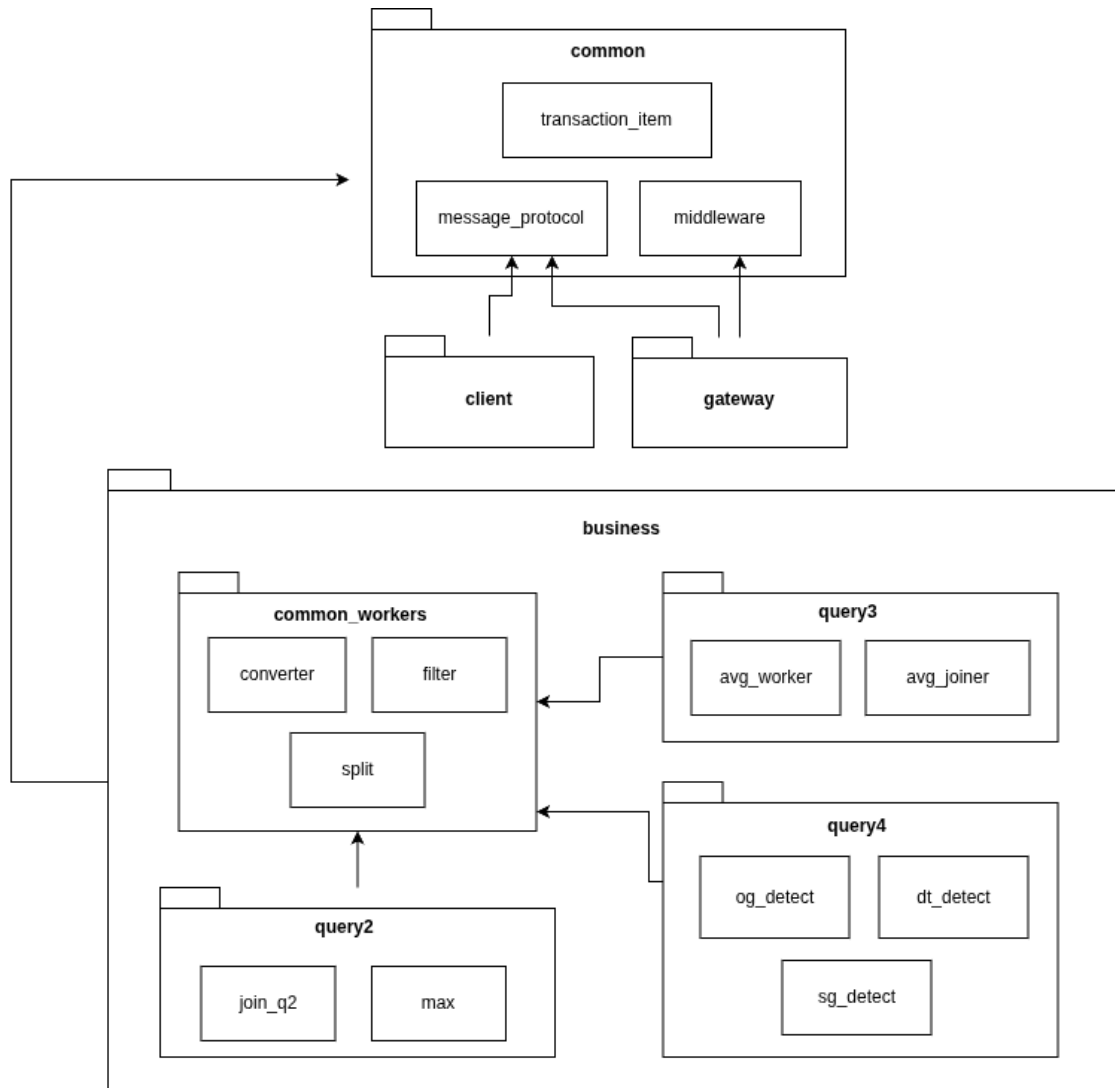
- `middleware` — abstrae la comunicación AMQP con el Message Broker.
- `message_protocol` — define la serialización y deserialización de mensajes entre workers.
- `transaction_item` — modela la transacción T_i y encapsula operaciones de comparación y validaciones de fecha y formato de pago.

Agrupamiento de workers

Los workers se agrupan dentro del paquete business por responsabilidad en cuatro subpaquetes:

- **common_workers** — workers genéricos y reutilizables entre consultas: converter, filter y split.
- **query2** — workers específicos de la consulta 2: max y join_q2.
- **query3** — workers específicos de la consulta 3: avg_worker y avg_joiner.
- **query4** — workers específicos de la consulta 4: og_detect, dt_detect y sg_detect.

Los subpaquetes query2, query3 y query4 dependen de common_workers.



Vista de procesos

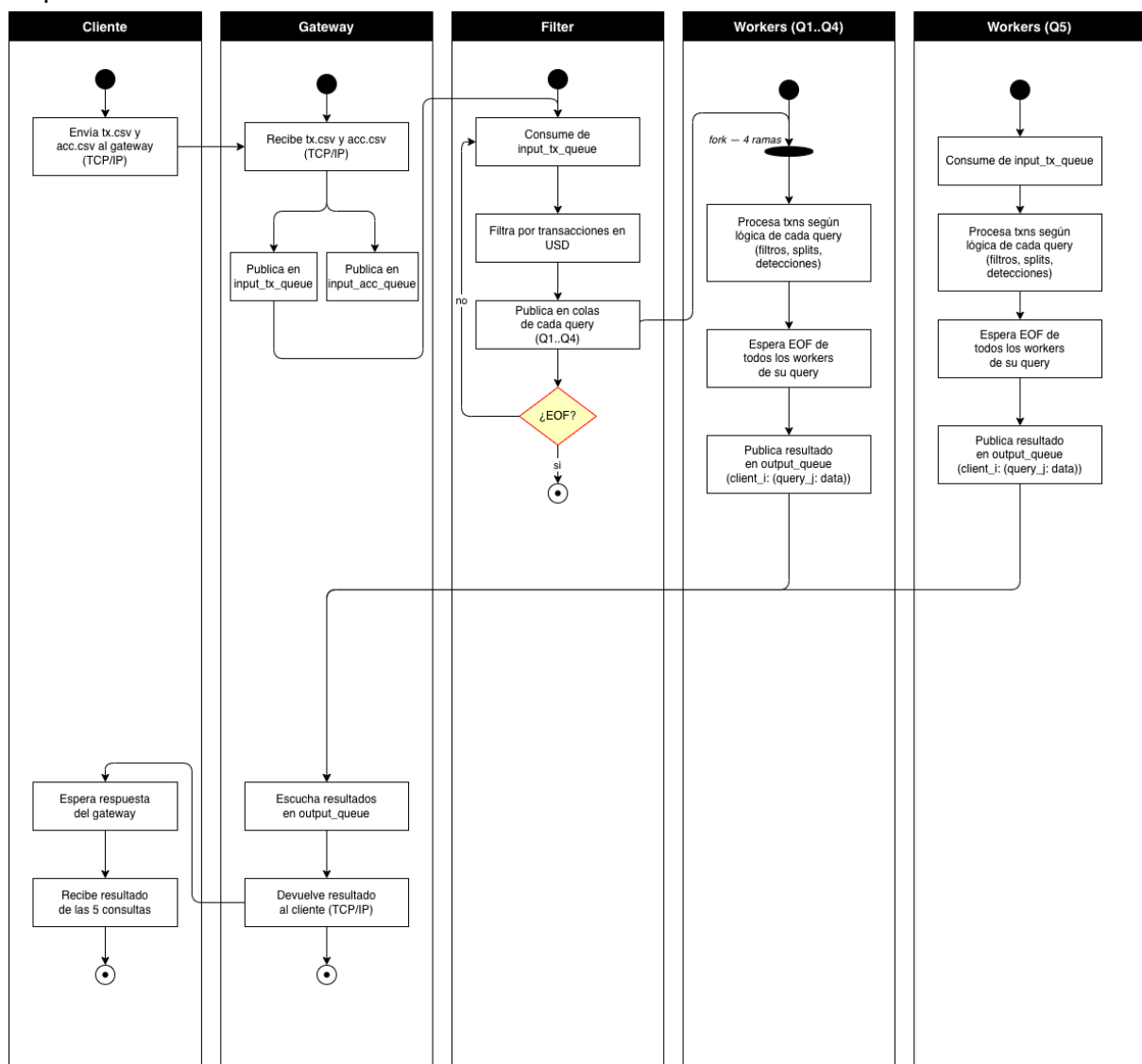
Diagrama de Actividades

El diagrama de actividades modela el flujo de control del sistema — qué pasos ocurren, en qué orden y bajo qué condiciones. A diferencia del diagrama de robustez que muestra qué componentes existen y cómo se conectan, este diagrama se enfoca en las decisiones y el paralelismo del procesamiento.

Flujo principal

El diagrama de actividades general modela el flujo de control del sistema completo, mostrando las interacciones entre los cinco participantes principales: Cliente, Gateway, Filter, Workers (Q1..Q5).

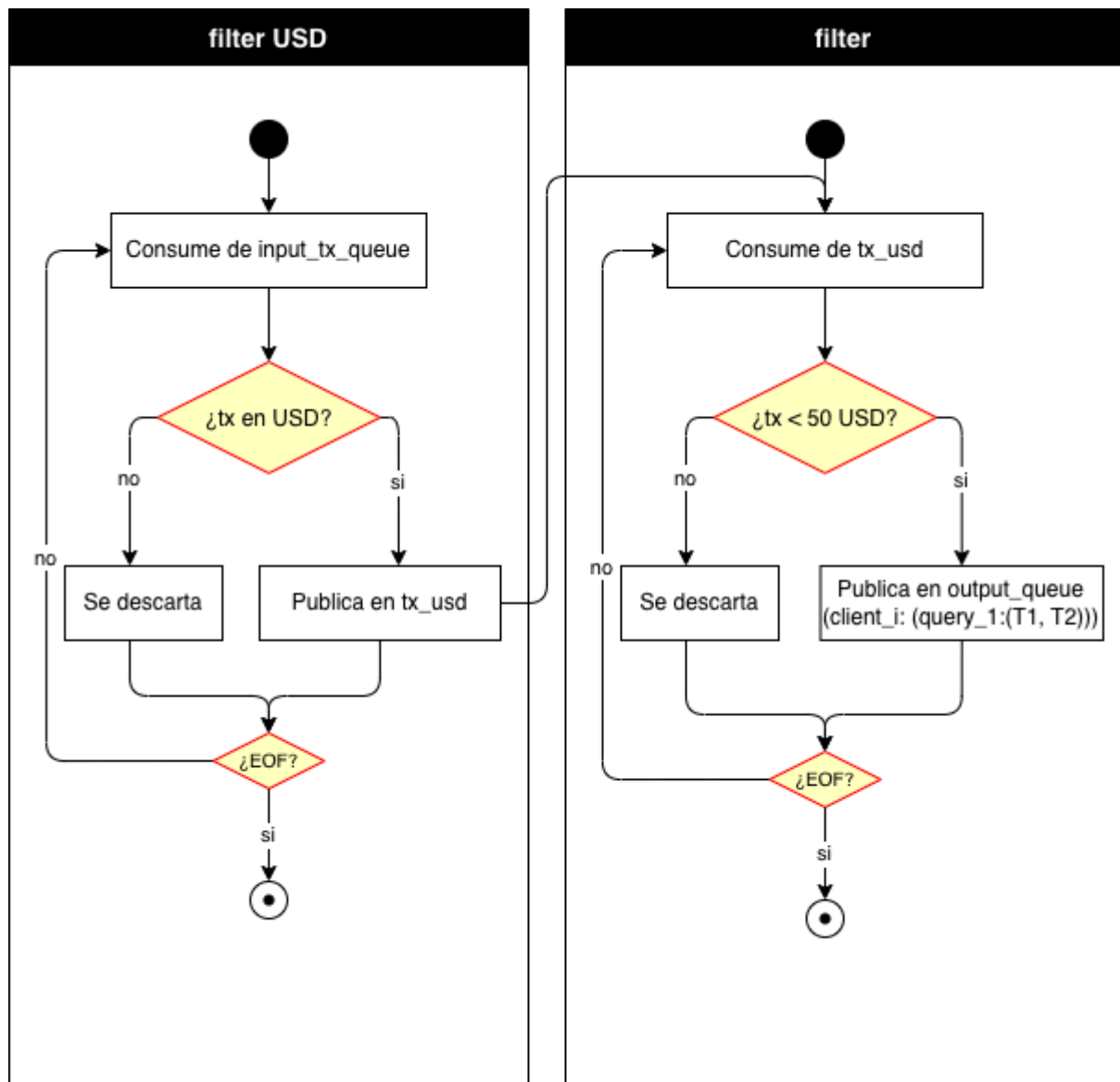
El cliente envía el archivo de transacciones al gateway, que las publica en RabbitMQ. El Filter descarta las transacciones que no son en USD y luego distribuye cada tx a las consultas correspondientes. Los workers de cada consulta procesan las tx en paralelo. En el caso de Q5, la query consume directamente del input sin filtrar. El converter realiza además el cambio a USD consultando Frankfurter API. Una vez que todas las consultas finalizan, los resultados son encolados en output_queue a medida que se van resolviendo y el gateway responde al cliente.



Query 1 — Transacciones USD menores a 50

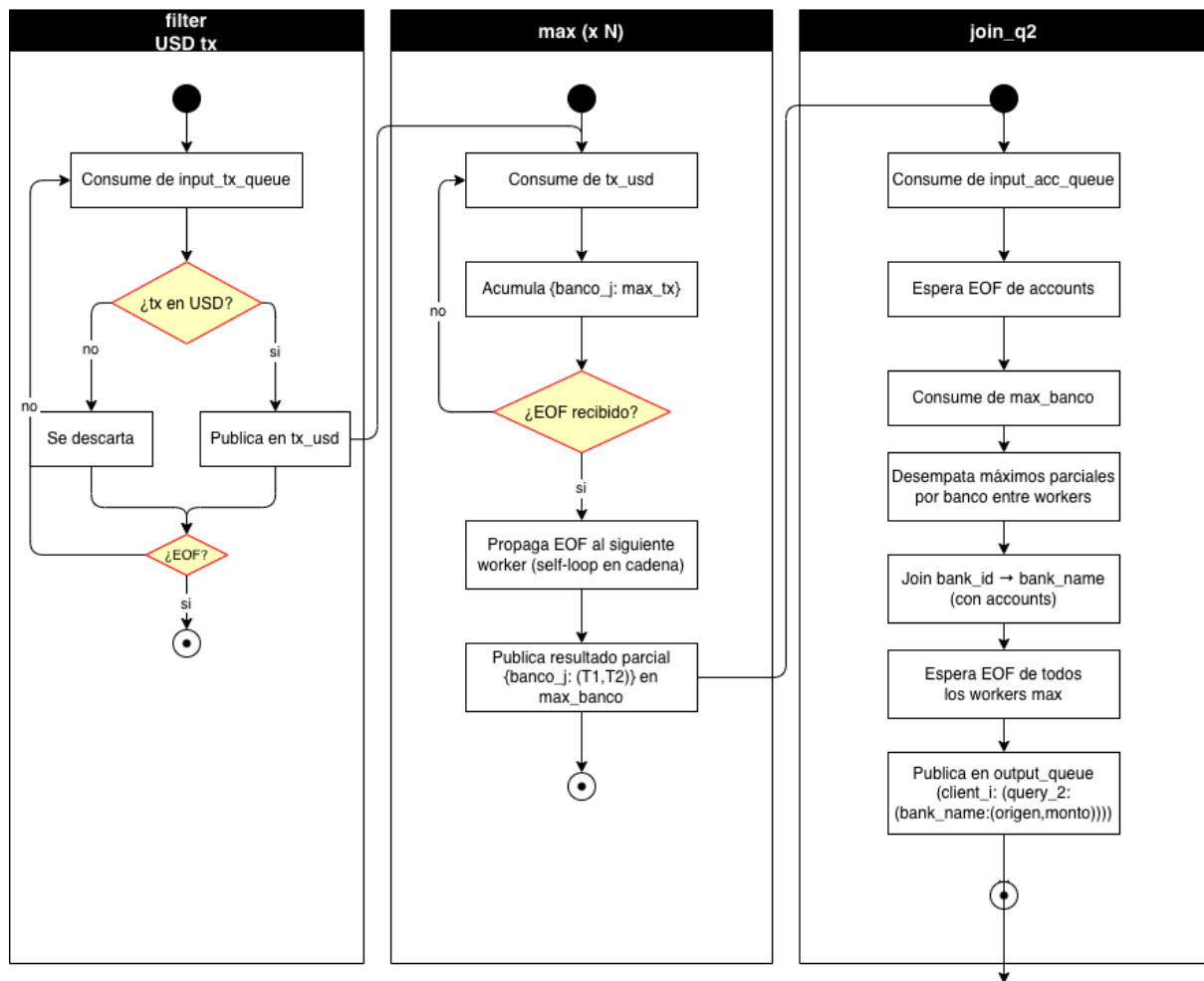
El diagrama tiene dos swimlanes en cadena. El filter USD consume de input_tx_queue y descarta las transacciones que no están en USD, publicando las que sí en tx_usd. El

segundo filter consume de tx_usd y descarta las que superan los 50 USD, publicando las que cumplen directamente en output_queue. Cada filter repite el loop hasta recibir el EOF.



Query 2 — Máxima transacción USD por banco

El diagrama tiene tres swimlanes: filter, max y join_q2. El filtro consume cada transacción, verifica si es en USD y la publica en tx_usd, descartando las que no son USD. Los workers max compiten por tx_usd acumulando el máximo por banco. Al recibir el EOF propagan el mensaje en cadena al siguiente worker mediante un self-loop antes de publicar su resultado parcial en max_banco. Este mecanismo garantiza que todos los workers emitan sus parciales antes de que el join consolide. El join_q2 opera en dos fases: primero consume de input_acc_queue y espera su EOF para tener todos los nombres de banco disponibles. Luego consume de max_banco, desempata los máximos parciales entre workers y hace el join de bank_id con bank_name. Al recibir el EOF de todos los workers max publica en output_queue.



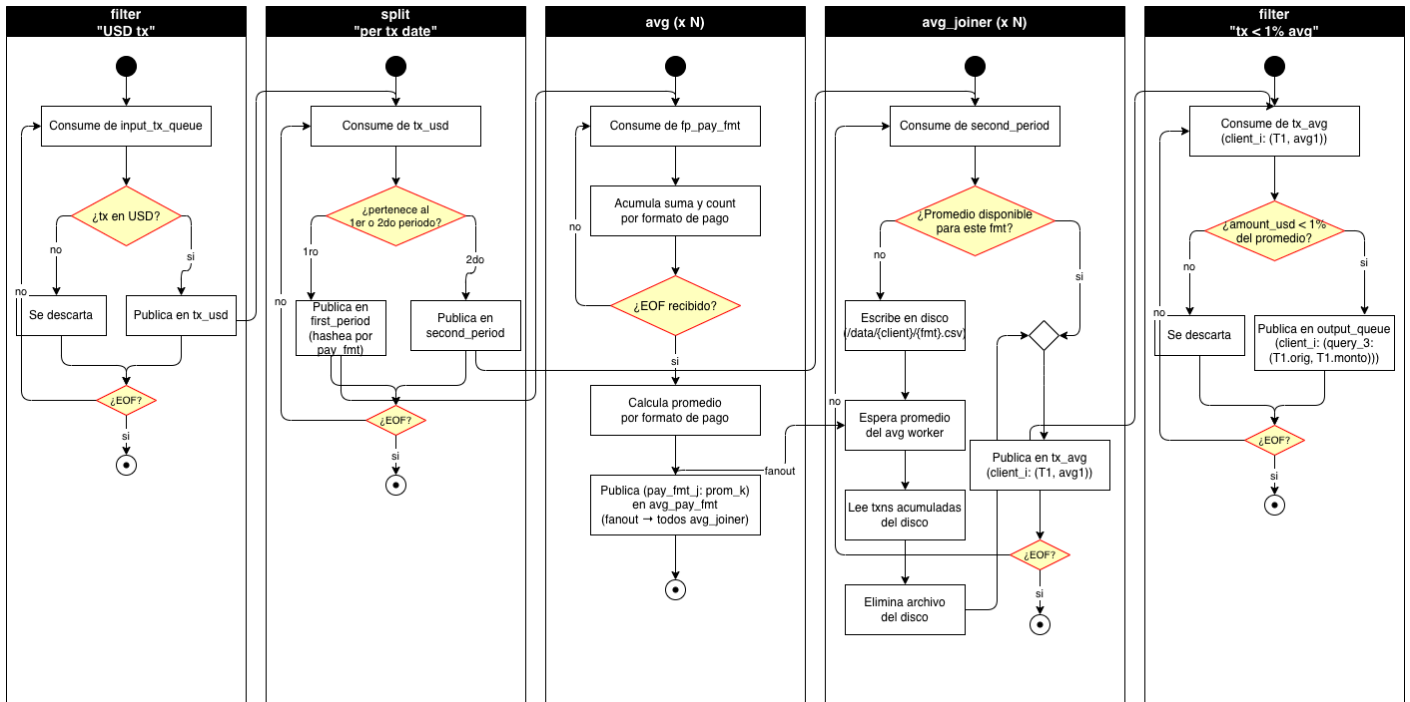
Query 3 — Transacciones menores al 1% del promedio del período anterior

El diagrama tiene cinco swimlanes: filter USD tx, split "per tx date", avg (x N), avg_joiner (x N) y filter "tx < 1% avg".

El filter consume de input_tx_queue, retiene sólo las transacciones en USD y las publica en tx_usd. El split consume de tx_usd y divide por fecha: las del período 1 las publica en first_period hasheando por formato de pago (para que las tx del mismo fmt siempre lleguen al mismo avg), las del período 2 las publica en second_period.

Los workers avg compiten por first_period, acumulan suma y count por formato de pago, y al EOF calculan el promedio y lo publican en avg_pay_fmt via fanout exchange, de modo que todos los avg_joiner reciben simultáneamente los promedios de todos los formatos de pago.

Los avg_joiner consumen de second_period. Para cada tx evalúan si el promedio de su formato ya llegó: si no llegó, escriben la tx en disco (/data/{client}/{fmt}.csv) y esperan; cuando llega via fanout, leen las txns del disco, eliminan el archivo y publican el par (client_i: (T1, avg1)) en tx_avg. El filter final consume de tx_avg, evalúa si amount_usd < 1% del promedio, publica las que cumplen directamente en output_queue y descarta las demás.



Query 4 — Patrón Scatter-Gather

El diagrama tiene seis swimlanes: filter_usd, filter_date, split, og_detect, dt_detect y sg_detect.

filter_usd consume transacciones de input_tx_queue y deja pasar solo las realizadas en USD, publicándolas en tx_usd.

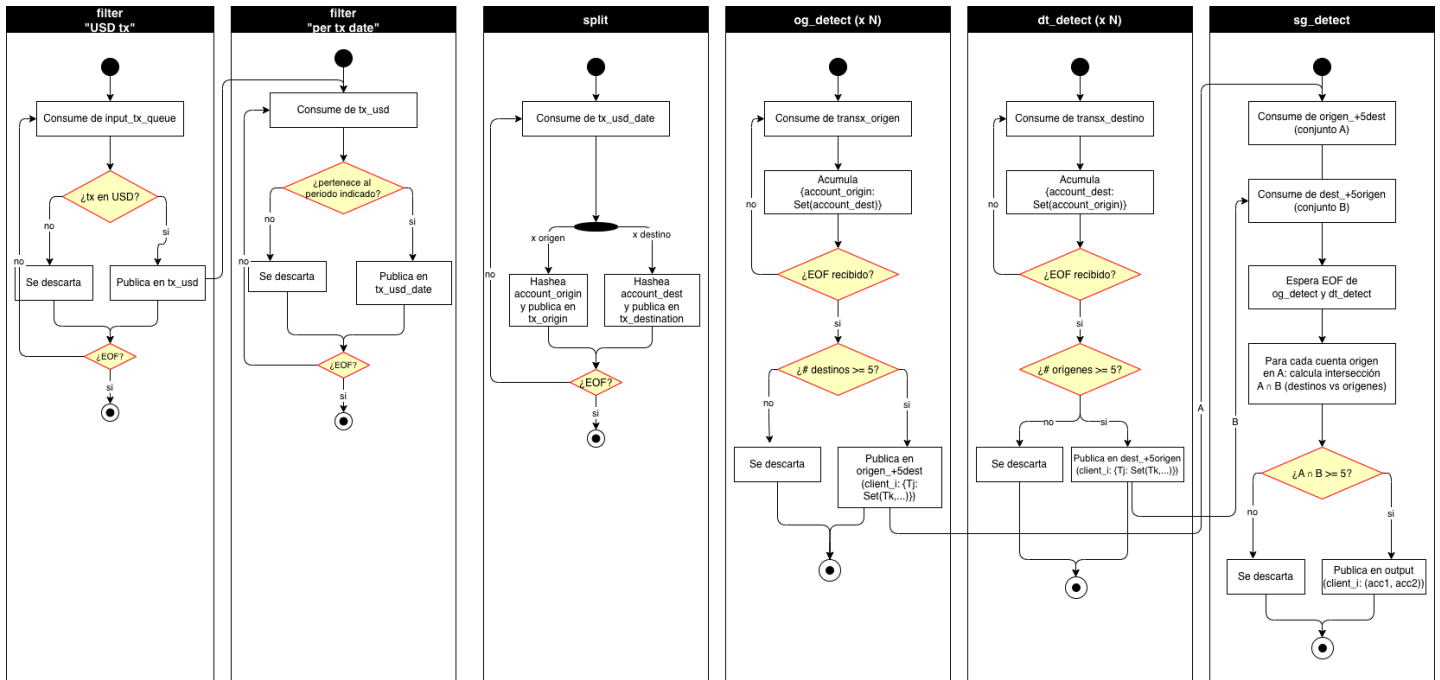
filter_date consume tx_usd y filtra las transacciones dentro del período indicado, publicándolas en tx_usd_date.

split toma cada transacción válida y hace un fork: hashlea account_origin y publica en transx_origin, y hashlea account_dest y publica en transx_destino. Así, todas las transacciones de una misma cuenta llegan siempre al mismo worker.

Los og_detect agrupan por cuenta origen y acumulan destinos distintos. Al EOF publican en origen_+5dest las cuentas con al menos 5 destinos.

Los dt_detect hacen lo mismo pero agrupando por cuenta destino y acumulando orígenes distintos, publicando en dest_+5origen.

Finalmente, sg_detect consume ambos conjuntos y verifica si una cuenta cumple el patrón scatter-gather mediante una intersección de tamaño al menos 5. Si cumple, publica el resultado en output; si no, se descarta.



Query 5 — Cantidad de transacciones Wire/ACH menores a 1 USD

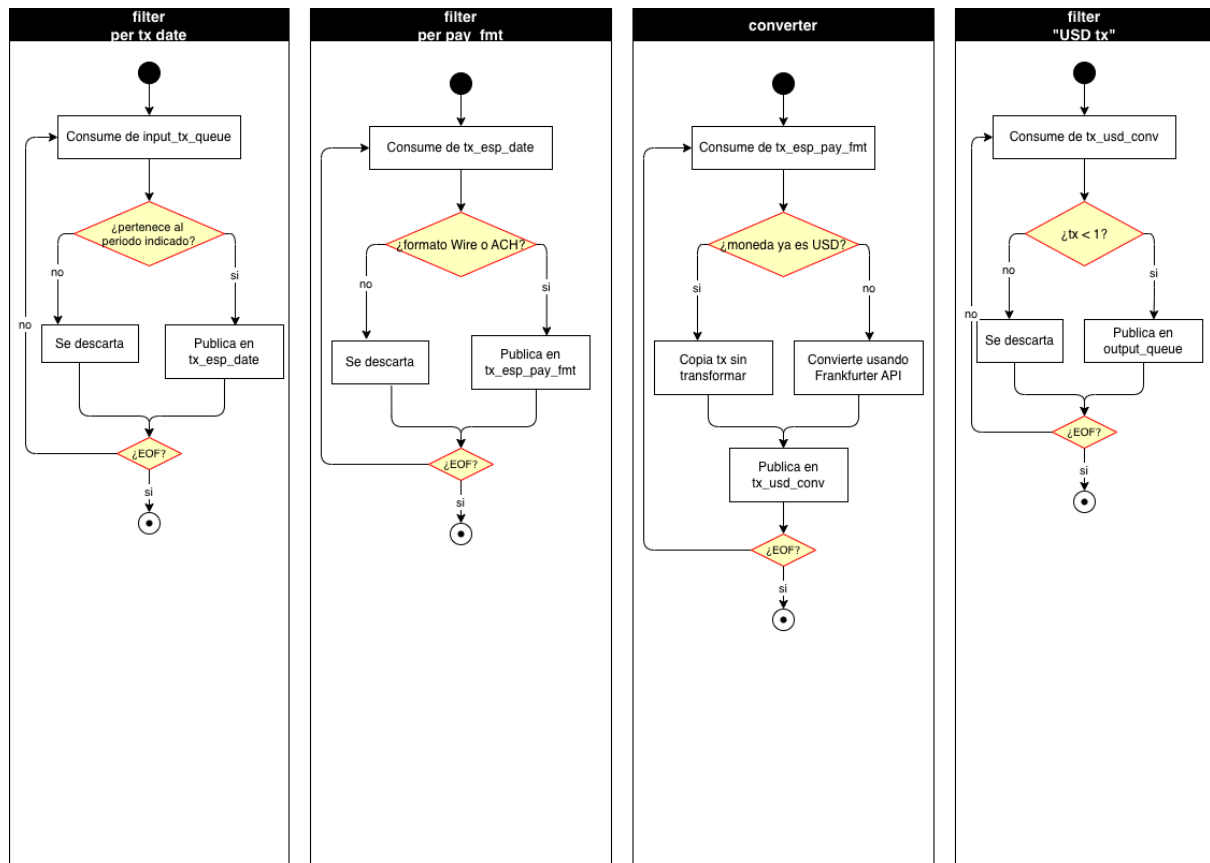
El diagrama tiene cuatro swimlanes: filter_date, filter_pay_fmt, converter y filter_usd.

filter_date consume transacciones de input_tx_queue y deja pasar solo las que pertenecen al período indicado, publicándolas en tx_esp_date.

filter_pay_fmt consume esas transacciones y filtra únicamente las que tienen formato de pago Wire o ACH, publicándolas en tx_esp_pay_fmt.

converter consume las transacciones filtradas y verifica si ya están en USD. Si no lo están, convierte el monto usando la API de Frankfurter. Luego publica el resultado en tx_usd_conv.

Finalmente, filter_usd consume desde tx_usd_conv y deja pasar únicamente las transacciones menores a 1 USD, publicándolas en output_queue. Las demás se descartan.

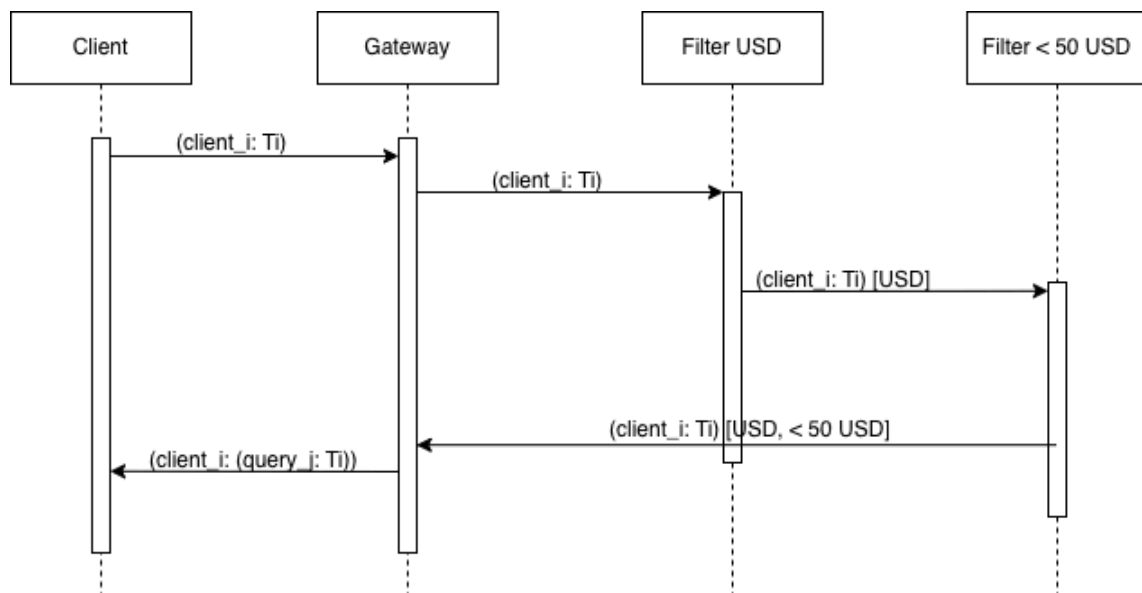


Diagramas de Secuencia

Los diagramas de secuencia modelan los mensajes entre participantes en orden temporal — quién le habla a quién, qué mensaje manda y cuándo. A diferencia del diagrama de actividades que muestra el flujo de control y decisiones, estos diagramas muestran el protocolo de comunicación concreto entre componentes.

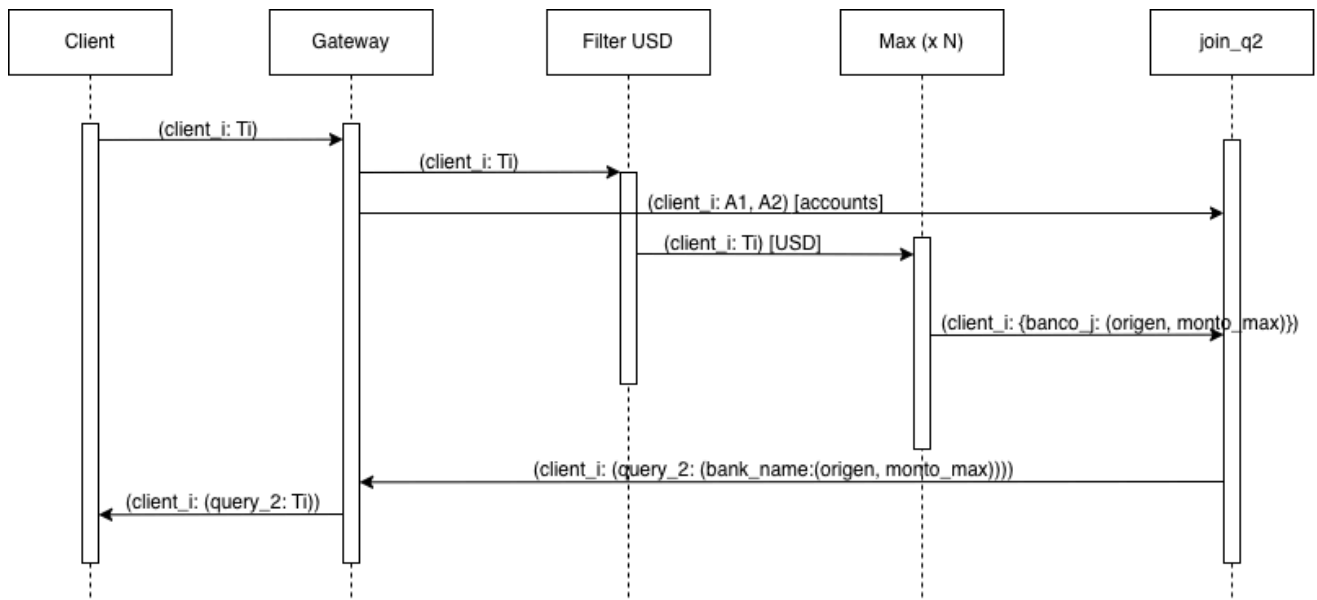
Consulta 1 - Transacciones USD menores a 50

El cliente envía las transacciones al gateway, que las reenvía al Filter USD. Este retiene solo las que están denominadas en USD y las pasa al Filter < 50. El segundo filter evalúa el monto y reenvía al gateway únicamente las transacciones cuyo valor es menor a 50 dólares. El gateway devuelve cada resultado al cliente a medida que llegan.



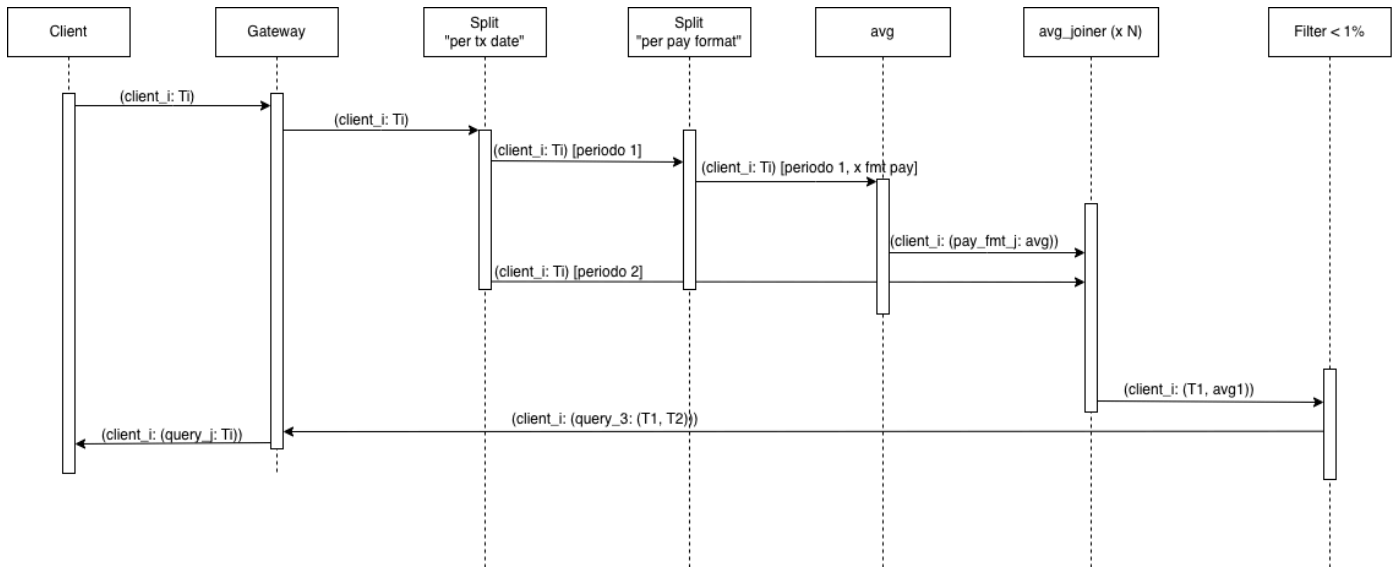
Consulta 2 - Máxima transacción USD por banco

El cliente envía tanto las transacciones como las cuentas bancarias. El gateway separa ambos flujos: las cuentas van directamente al join y las transacciones pasan por el Filter USD. Los workers Max compiten por las transacciones USD, acumulan el máximo por banco y al EOF publican su parcial {banco_j: (origen, monto_max)} en join. Este, que ya tiene los nombres de banco de las accounts, desempata los parciales, hace el join y devuelve el monto máximo de transacción por cada banco junto con su correspondiente nombre.



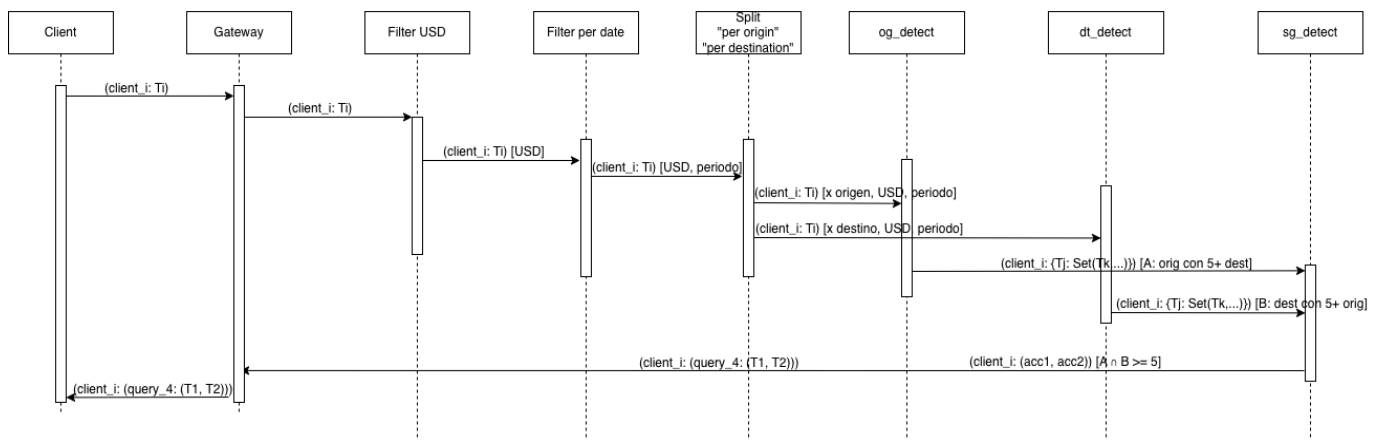
Consulta 3 - Transacciones menores al 1% del promedio

El split divide las transacciones USD por fecha en dos ramas paralelas. Las del período 1 se redistribuyen por formato de pago hacia los workers avg, que calculan el promedio y lo difunden via fanout a todos los avg_joiner. Las del período 2 son consumidas por los avg_joiner, que las persisten en disco mientras esperan el promedio correspondiente. Al recibirlo, procesan las transacciones almacenadas y las envían al filter, que publica en output_queue las que tienen amount_usd < 1% del promedio.



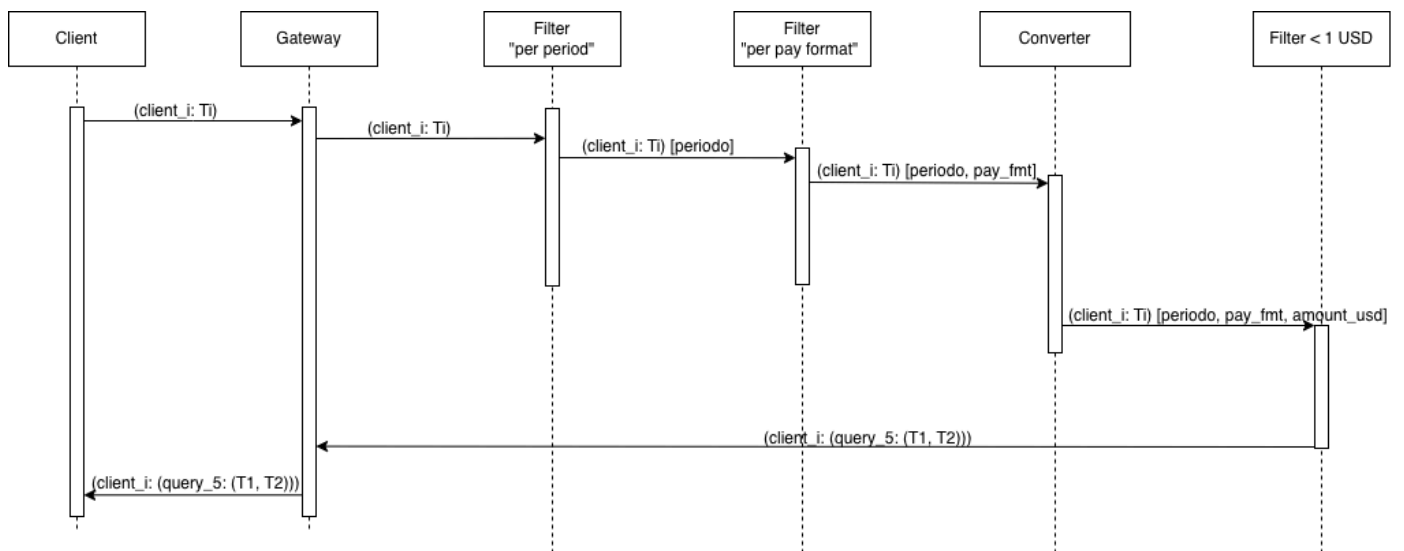
Consulta 4 - Patrón Scatter-Gather

Las transacciones pasan por Filter USD y luego por Filter per date, quedando solo las del período indicado en USD. El Split hace un fork por cada transacción: la publica simultáneamente hasheando por origen hacia og_detect y por destino hacia dt_detect. El og_detect emite las cuentas origen con 5 o más destinos distintos (conjunto A) y el dt_detect las cuentas destino con 5 o más orígenes distintos (conjunto B). El sg_detect recibe ambos conjuntos, calcula la intersección y publica los pares de cuentas donde $A \cap B \geq 5$.



Consulta 5 - Cantidad de transacciones Wire/ACH menores a 1 USD

Las transacciones pasan primero por filtrado de periodo, que retiene sólo las del período indicado, y luego por filtrado de formato de pago, que retiene sólo las Wire o ACH. El converter recibe cada transacción y consulta la Frankfurter API para obtener el tipo de cambio y calcular amount_usd. Luego se descartan las transacciones el monto de 1 USD y el gateway devuelve el conteo total al cliente.



Tolerancia a Fallos

Watchdog y heartbeats

Cada worker corre un hilo (HeartbeatSender) que cada tanto publica un mensaje a Rabbit con su nombre de container y timestamp. Estos mensajes se envían a todos los watchdogs por medio del exchange heartbeat_exchange.

Los watchdogs actualizan un diccionario last_seen: {container_name: timestamp} cada vez que llega un heartbeat. Un revive loop corre cada REVIVE_INTERVAL segundos y, si detecta que algún worker no mandó heartbeat en más de HEARTBEAT_TIMEOUT segundos, lo reinicia usando la API de Docker.

Para que el sistema de watchdogs sea en sí mismo tolerante a fallos, se despliegan múltiples instancias. Los watchdogs se vigilan entre sí mediante un exchange separado (peer_exchange), formando un anillo: el watchdog N monitorea al watchdog N-1 (circular). Si un watchdog no recibe heartbeat de su par en WATCHDOG_TIMEOUT segundos, lo reinicia.

Para evitar que múltiples watchdogs intenten reiniciar los mismos workers simultáneamente, se utiliza una elección de líder implícita por ID: solo el watchdog vivo con el ID más bajo actúa como líder y ejecuta la revisión de workers. Los demás watchdogs siguen mandándose heartbeats entre sí, pero omiten la verificación de workers mientras haya un líder activo.

Checkpoints y deduplicación

Dado que RabbitMQ ofrece garantías de entrega at-least-once, el watchdog puede reiniciar un worker y RabbitMQ puede reentregarle el último mensaje que no fue confirmado. Para evitar procesar ese mensaje dos veces, cada worker guarda un checkpoint con el hash del último mensaje procesado. Al arrancar, carga el checkpoint y descarta cualquier mensaje cuyo hash coincida con el ya registrado.

Los checkpoints también preservan el estado interno relevante del worker ante una caída. Por ejemplo, el filter persiste el conjunto eof_seen (clientes cuyo EOF ya fue propagado downstream) para no reenviar EOFs duplicados si es reiniciado.

Disk spill atómico y recuperación

Los workers que acumulan filas antes de enviarlas (filter, avg_joiner) nunca escriben directamente en el archivo final. El flujo es:

1. Escribir las filas nuevas en un archivo temporal .tmp
2. Hacer os.replace(tmp, path) — operación atómica en Linux

Esto garantiza que el archivo nunca quede en estado corrupto ante una caída en medio de la escritura. Lo peor que puede ocurrir es que quede el .tmp huérfano, mientras el archivo principal permanece intacto.

Al reiniciarse, cada worker recorre su directorio de datos (/data/<client_id>/rows.csv) y reenvía cualquier archivo que haya quedado escrito en disco pero no enviado a RabbitMQ. Esto cubre el caso en que el worker cayó después de escribir en disco pero antes de confirmar el mensaje en RabbitMQ.

Garantía resultante

La combinación de estos tres mecanismos produce una garantía de procesamiento *effectively exactly-once*:

- RabbitMQ entrega *at-least-once*
- El checkpoint con deduplicación por hash elimina el doble procesamiento
- El disk spill atómico con recovery al inicio elimina la pérdida de datos ante caídas entre escritura y envío

Backlog

Tarea	Descripción	Assignee	Puntos
Implementar common/middleware.py	Abstracción AMQP para consumir y publicar mensajes en RabbitMQ	V	1
Implementar common/message_protocol.py	Serialización y deserialización de mensajes entre workers	V	1
Implementar common/transaction_item.py	Modelo Ti, comparaciones y validaciones de fecha y formato de pago	V - T	1
Implementar gateway	Servidor TCP/IP, recibe tx de clientes y publica en input_queue	V - T	1
Armado de docker-compose	Orquesta todos los containers, configura redes y volúmenes	V - T	1
Armado de Dockerfiles	Dockerfile por cada worker y container del sistema	V - T	1
Implementar filter	Filtra tx por criterio específico — moneda, fecha, formato de pago o monto	V - T	2
Implementar converter	Convierte monto de cada tx a USD consultando la Currency Exchange API	V	1
Implementar max	Acumula max tx por banco, emite resultado al EOF via self-loop	-	-
Implementar split	Divide flujo de tx por fecha para Q3 y por origen/destino para Q4	V - T	2
Implementar avg	Calcula promedio de tx del período 1 agrupado por formato de pago	V	3
Implementar og_detect	Acumula {origen: Set<destinos>}, emite orígenes con 5+ destinos al EOF	T	2
Implementar dt_detect	Acumula {destino: Set<orígenes>}, emite destinos con 5+ orígenes al EOF	T	2
Implementar avg_joiner	Combina tx período 2 con promedio período 1, incluye lógica de spill to disk	V	3
Implementar sg_detect	Calcula intersección entre conjuntos de og_detect y dt_detect, confirma patrón scatter-gather	T	3
Implementar join_q2	Desempata máximos parciales entre workers max y hace join con accounts	-	-
Tests E2E	Verifican el sistema completo levantando el docker-compose y validando los 5 resultados contra los valores del notebook patrón	T - V	